Sistema de Gestión para Consultorio Médico “Salud Integral”

Análisis de requisitos, diseño con el patrón Repository y prototipo web

Informe técnico

José Ignacio Gómez Guzmán
[Integrante 2]
[Integrante 3]
[Integrante 4]
[Integrante 5]

Líder del equipo: [Nombre del líder del equipo]

[Nombre de la universidad]
[Nombre del curso]
Docente: [Nombre del docente]

25 de octubre de 2026

Repositorio: https://github.com/JNACHO2001/consultorio-medico

Aplicación en línea: https://consultorio-medico-pty2.onrender.com

Contenido

1. Introducción **3**

2. Descripción del problema y objetivos **3**

2.1 Situación actual 3

2.2 Declaración del problema 3

2.3 Objetivos 4

3. Levantamiento de requisitos **4**

3.1 Partes interesadas y alcance 4

3.2 Requisitos funcionales 5

3.3 Requisitos no funcionales 5

3.4 Reglas de negocio 6

4. Modelado con UML **6**

4.1 Casos de uso 6

4.2 Clases de dominio 7

4.3 Clases: servicios y patrón Repository 8

4.4 Despliegue 9

4.5 Secuencia: agendar una cita 10

5. Patrones de diseño **12**

5.1 Repository: el patrón principal 12

5.2 Arquitectura por capas y composición de dependencias 13

5.3 Patrones de nube y seguridad aplicados 13

6. Prototipo **13**

6.1 Tecnología y estructura 13

6.2 API REST 13

6.3 Seguridad y reglas de negocio 14

6.4 Interfaz de usuario 14

6.5 Pruebas y resultados 17

6.6 Cobertura de requisitos y límites 18

6.7 Despliegue 18

7. Conclusiones **18**

Referencias **19**

1. Introducción

Los consultorios médicos pequeños suelen guardar la información de sus pacientes en carpetas de papel y hojas de cálculo sueltas. Ese modo de trabajo funciona mientras el volumen es bajo, pero expone al consultorio a perder historias clínicas, duplicar pacientes, cruzar citas y mostrar datos sensibles a quien no debería verlos. En Colombia, además, la información clínica está protegida por la Ley Estatutaria 1581 de 2012 (Congreso de la República de Colombia, 2012) y por la Resolución 1995 de 1999 sobre historia clínica (Ministerio de Salud de Colombia, 1999), de modo que su manejo descuidado tiene consecuencias legales.

Este informe documenta el análisis, el diseño y la construcción de un prototipo de sistema web para un consultorio ficticio, “Salud Integral”, que atiende cerca de 60 pacientes al día con tres médicos generales, un especialista, dos recepcionistas y un administrador. El objetivo es centralizar y proteger los datos de pacientes, citas e historias clínicas, y demostrar cómo el patrón de diseño *Repository* (Fowler, 2002; Evans, 2003) ayuda a controlar en un solo lugar las reglas que más importan: unicidad del paciente, borrado lógico y auditoría de accesos.

El documento sigue el orden de trabajo del proyecto. La sección 2 describe el problema y los objetivos; la sección 3 resume el levantamiento de requisitos; la sección 4 presenta los diagramas UML; la sección 5 explica los patrones de diseño elegidos y por qué; la sección 6 describe el prototipo construido con Python (FastAPI) y JavaScript, sus pruebas y sus límites; y la sección 7 reúne las conclusiones. El código fuente completo, los diagramas y las instrucciones para ejecutarlo están en el repositorio del proyecto.

2. Descripción del problema y objetivos

2.1 Situación actual

Hoy la recepcionista anota las citas en un cuaderno o por teléfono, busca la carpeta física del paciente al llegar (y crea otra si no la encuentra, sin verificar si ya existía), y el médico escribe la consulta a mano y la devuelve al archivo. Algunos datos se copian después en hojas de Excel con formatos distintos según quién las llene. No hay copia de seguridad: todo depende de un archivador y de un solo computador. Esta situación se levantó mediante análisis del caso de estudio, entrevistas guiadas, observación del flujo de atención y revisión de los documentos existentes (Wiegers y Beatty, 2013).

2.2 Declaración del problema

Salud Integral guarda hoy la información de sus pacientes en carpetas de papel y hojas de Excel sueltas, sin respaldo ni control de acceso, por lo que ya perdió datos y no puede garantizar su confidencialidad. Afecta a pacientes, médicos, recepcionistas, administrador y propietario; su impacto es pérdida de información clínica, historiales fragmentados, citas cruzadas, búsquedas lentas e incumplimiento de la protección de datos personales. Una solución exitosa sería un sistema web en la nube que centralice la información, impida duplicados, controle quién ve qué, registre cada acceso y se respalde automáticamente.

Tabla 1

Dolores identificados en el consultorio

ID	Dolor	Causa raíz	Severidad
D-01	Las historias clínicas en papel se extravían o deterioran	Almacenamiento físico sin copia	Crítica
D-02	Pacientes duplicados con datos distintos	No se valida el documento al crear la carpeta	Alta
D-03	Citas cruzadas e inasistencias	Agenda en cuaderno, sin recordatorios	Alta
D-04	No hay copias de seguridad	Datos en papel y en un solo computador	Crítica
D-05	Cualquier persona puede ver cualquier historia	Sin permisos por rol	Crítica
D-06	Buscar un historial toma varios minutos	Información dispersa y sin índice	Media
D-07	No se sabe quién consultó o modificó un dato	No existe registro de accesos	Alta
D-08	Datos desorganizados y con formatos distintos	Texto libre sin campos obligatorios	Media

Nota. Tres dolores son críticos (D-01, D-04 y D-05) porque comprometen la continuidad del servicio o el cumplimiento legal.

2.3 Objetivos

Objetivo general. Diseñar y construir un sistema web que centralice, organice y proteja la información de pacientes, citas e historias clínicas de Salud Integral, evitando la pérdida de datos.

Objetivos específicos. (a) Centralizar los datos de los pacientes con identificación única para eliminar duplicados; (b) digitalizar la agenda con validación de horarios y recordatorios; (c) registrar y consultar la historia clínica de forma estructurada y rápida; (d) controlar el acceso por roles y registrar toda acción en una bitácora; (e) garantizar respaldos y recuperación ante fallos en la nube; y (f) aislar el acceso a datos mediante el patrón Repository para facilitar el mantenimiento y las pruebas.

3. Levantamiento de requisitos

Los requisitos se redactaron de forma verificable, con criterio de aceptación y prioridad según la técnica MoSCoW, siguiendo las recomendaciones de la norma ISO/IEC/IEEE 29148 (2018) y de Sommerville (2016). El documento completo de levantamiento incluye además la descripción detallada del problema, la matriz de trazabilidad entre dolores, requisitos y casos de uso, y los riesgos del proyecto; aquí se resume lo esencial.

3.1 Partes interesadas y alcance

Siete partes interesadas intervienen o se ven afectadas; cuatro interactúan con el sistema como actores: la **recepcionista** (registra pacientes y agenda citas), el **médico** (atiende y consulta historias clínicas), el **administrador** (gestiona usuarios y revisa la bitácora) y el **servicio de notificaciones** (sistema externo que envía recordatorios). El sistema cubre seis módulos: gestión de pacientes, agenda de citas, historia clínica, seguridad y usuarios, auditoría y notificaciones. Quedan fuera de esta versión la facturación, el inventario, la integración con aseguradoras, la telemedicina y la aplicación móvil nativa.

3.2 Requisitos funcionales

Tabla 2

Requisitos funcionales

ID	Requisito	Actor	Prioridad
RF-01	Registrar un paciente con documento, nombres, fecha de nacimiento, sexo, contacto, EPS y alergias	Recepción	Must
RF-02	Impedir pacientes duplicados validando tipo y número de documento	Sistema	Must
RF-03	Buscar pacientes por documento o por nombre, incluso con texto parcial (menos de 2 s)	Recepción, médico	Must
RF-04	Actualizar datos del paciente conservando el historial de cambios	Recepción	Should
RF-05	Agendar, reprogramar y cancelar citas sin cruces de horario del médico	Recepción	Must
RF-06	Mostrar la agenda diaria y semanal por médico	Recepción, médico	Must
RF-07	Registrar una consulta (motivo, antecedentes, signos vitales, diagnóstico, tratamiento, fórmula); una consulta cerrada solo se corrige con nota aclaratoria	Médico	Must
RF-08	Consultar el historial clínico completo, de la consulta más reciente a la más antigua	Médico	Must
RF-09	Adjuntar documentos (PDF, JPG, PNG de hasta 10 MB) a una consulta	Médico	Should
RF-10	Enviar un recordatorio de la cita 24 horas antes	Servicio de notificaciones	Should
RF-11	Crear y desactivar usuarios y asignar roles	Administrador	Must
RF-12	Autenticar con usuario y contraseña y cerrar la sesión por inactividad	Todos	Must
RF-13	Registrar quién consultó o modificó cada dato clínico y cuándo; consulta para el administrador	Administrador	Must
RF-14	Generar reportes básicos de citas por día y pacientes atendidos por médico	Administrador	Could

Nota. Prioridad según MoSCoW: Must (debe), Should (debería), Could (podría).

3.3 Requisitos no funcionales

Tabla 3

Requisitos no funcionales

ID	Categoría	Requisito y métrica de verificación
RNF-01	Seguridad	Acceso limitado por rol; la recepción no puede ver diagnósticos (0 accesos indebidos)
RNF-02	Seguridad	Contraseñas almacenadas con hash; toda comunicación por HTTPS
RNF-03	Privacidad	Cumplir la Ley 1581 de 2012 y la Resolución 1995 de 1999
RNF-04	Disponibilidad	99 % o más en el horario de atención, medido con un endpoint de salud
RNF-05	Confiabilidad	Copias de seguridad diarias; pérdida máxima de 24 h; restauración en menos de 4 h
RNF-06	Rendimiento	Búsqueda de pacientes en menos de 2 s con 10 000 pacientes
RNF-07	Escalabilidad	Aumentar capacidad sin rediseñar la arquitectura
RNF-08	Usabilidad	Un recepcionista nuevo completa un registro y una cita tras menos de 1 h de capacitación
RNF-09	Mantenibilidad	Arquitectura por capas con acceso a datos aislado mediante Repository
RNF-10	Portabilidad	Funcionar desde un navegador, sin instalación local
RNF-11	Integridad	Los datos clínicos no se eliminan físicamente (borrado lógico)

Nota. Las metas de recuperación y disponibilidad son propuestas del equipo y deben validarse con el consultorio.

3.4 Reglas de negocio

Siete reglas gobiernan el sistema: **RN-01**, un paciente se identifica de forma única por su tipo y número de documento; **RN-02**, un médico no puede tener dos citas en el mismo horario; **RN-03**, solo el médico crea y ve el contenido clínico; **RN-04**, la historia clínica no se elimina ni se altera y las correcciones se hacen con una nota aclaratoria fechada y firmada; **RN-05**, toda consulta o modificación de datos clínicos queda en la bitácora; **RN-06**, al cancelar una cita el horario queda libre de inmediato; y **RN-07**, los datos personales solo se usan para fines asistenciales y administrativos.

4. Modelado con UML

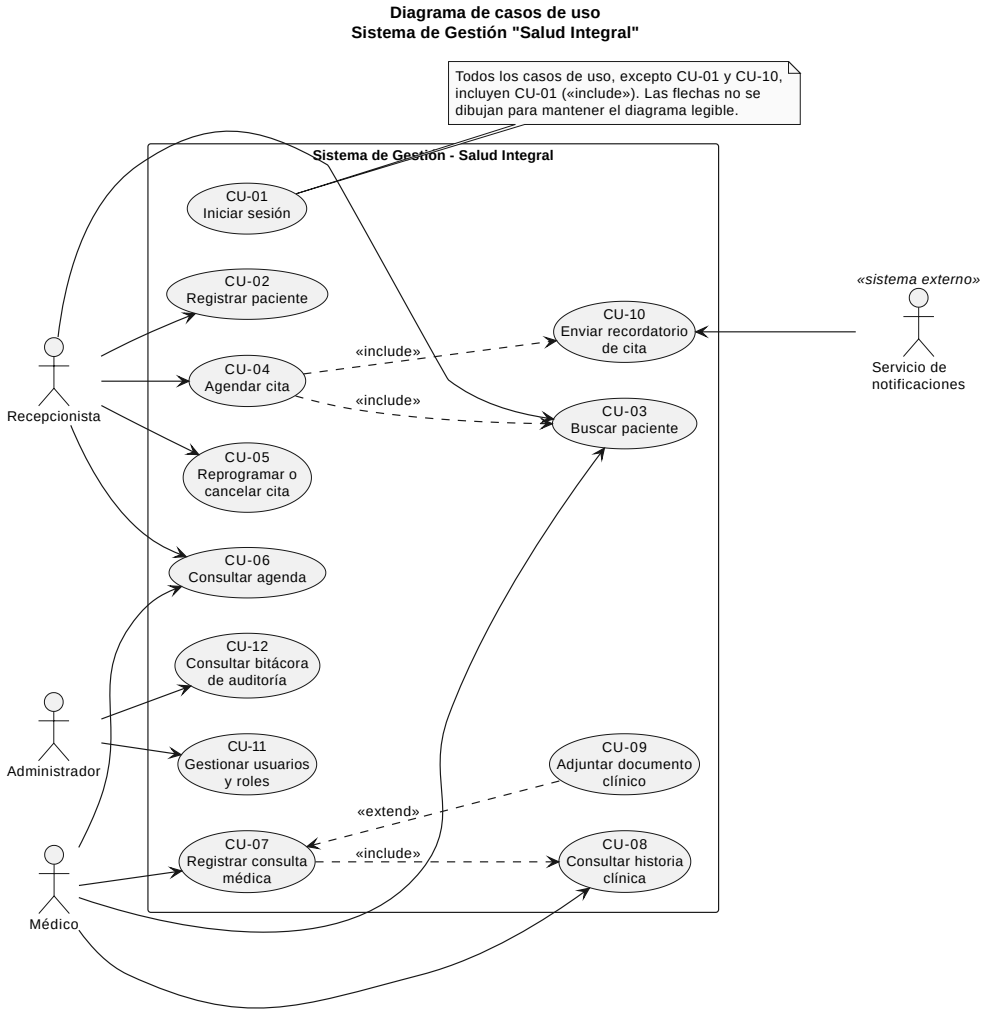
Los diagramas se elaboraron con PlantUML a partir de los requisitos y se mantienen como texto en el repositorio (carpeta `docs/uml`), de modo que cambian junto con el código. Se eligieron cinco vistas complementarias: qué hace el sistema (casos de uso), con qué conceptos trabaja (clases de dominio), cómo se organiza el acceso a datos (Repository), dónde se ejecuta (despliegue) y cómo colaboran los componentes en el flujo más importante (secuencia).

4.1 Casos de uso

Se identificaron 12 casos de uso que cubren los 14 requisitos funcionales (Figura 1). Todos, salvo el inicio de sesión y el envío de recordatorios, incluyen CU-01 *Iniciar sesión*, lo que expresa que ninguna función es accesible sin autenticación. Agendar una cita (CU-04) incluye buscar al paciente (CU-03) y enviar el recordatorio (CU-10); registrar una consulta (CU-07) incluye consultar el historial (CU-08), y adjuntar un documento (CU-09) la extiende de forma opcional.

Figura 1

Diagrama de casos de uso



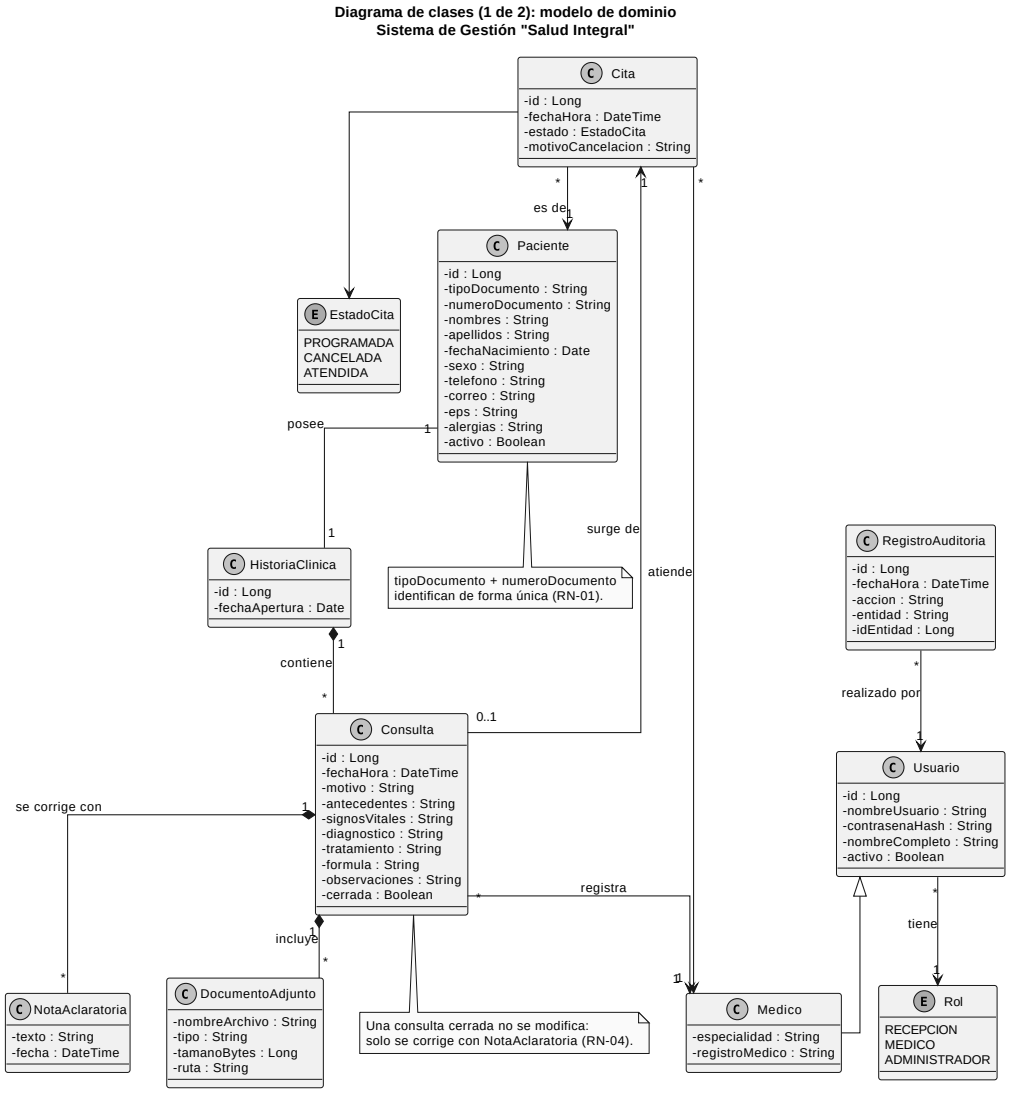
Nota. Elaboración propia con PlantUML. Fuente: `docs/uml/01_casos_de_uso.puml`.

4.2 Clases de dominio

El modelo de dominio (Figura 2) reúne las entidades que el consultorio necesita recordar: *Paciente*, *Cita*, *Consulta*, *NotaAclaratoria*, *Usuario* (con sus especializaciones *Médico* y los roles) y *RegistroAuditoria*. Todas heredan de una entidad base con identificador y estado activo, que es lo que permite el borrado lógico exigido por RNF-11. La consulta está asociada a una cita, un paciente y un médico, y sus correcciones se guardan como notas aclaratorias en lugar de editar el texto original (RN-04).

Figura 2

Diagrama de clases de dominio



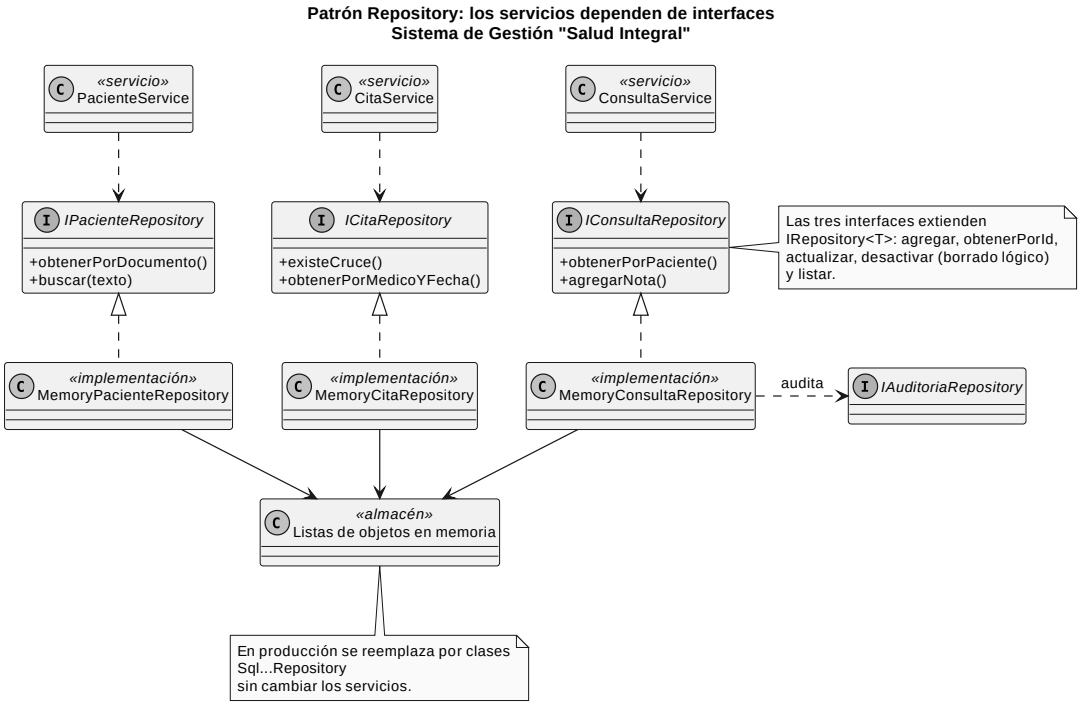
Nota. Elaboración propia con PlantUML. Fuente: `docs/uml/02a_clases_dominio.puml`.

4.3 Clases: servicios y patrón Repository

La Figura 3 muestra la separación en capas. Los servicios de aplicación contienen la lógica de negocio y dependen únicamente de interfaces (*IPacienteRepository*, *ICitaRepository*, *IConsultaRepository*); las clases *Memory…Repository* implementan esas interfaces y son las únicas que tocan el almacén de datos. Los repositorios de pacientes y consultas, además, anotan cada acceso a través de *IAuditoriaRepository*. La vista completa, con los seis repositorios y todas sus operaciones, está en `docs/uml/02b_clases_repository.puml`.

Figura 3

Patrón Repository: servicios, interfaces e implementaciones



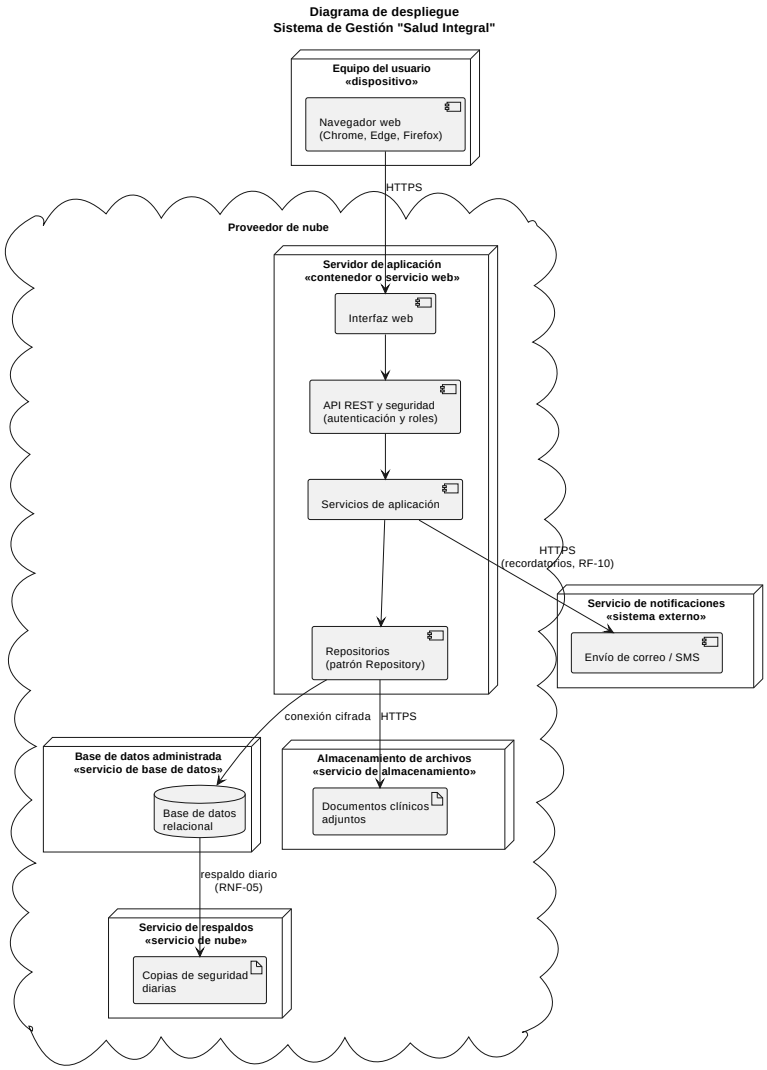
Nota. Vista resumida con tres de los seis repositorios; los de usuarios, recordatorios y auditoría siguen el mismo esquema.

4.4 Despliegue

La Figura 4 muestra la arquitectura de ejecución prevista: el navegador del personal se comunica por HTTPS con una aplicación contenerizada en un servicio de nube, que a su vez usa el almacén de datos y un servicio de notificaciones. En el prototipo el almacén son listas de objetos en memoria y el envío de correos se simula; el diagrama conserva el diseño objetivo, que solo exige sustituir las implementaciones de los repositorios.

Figura 4

Diagrama de despliegue



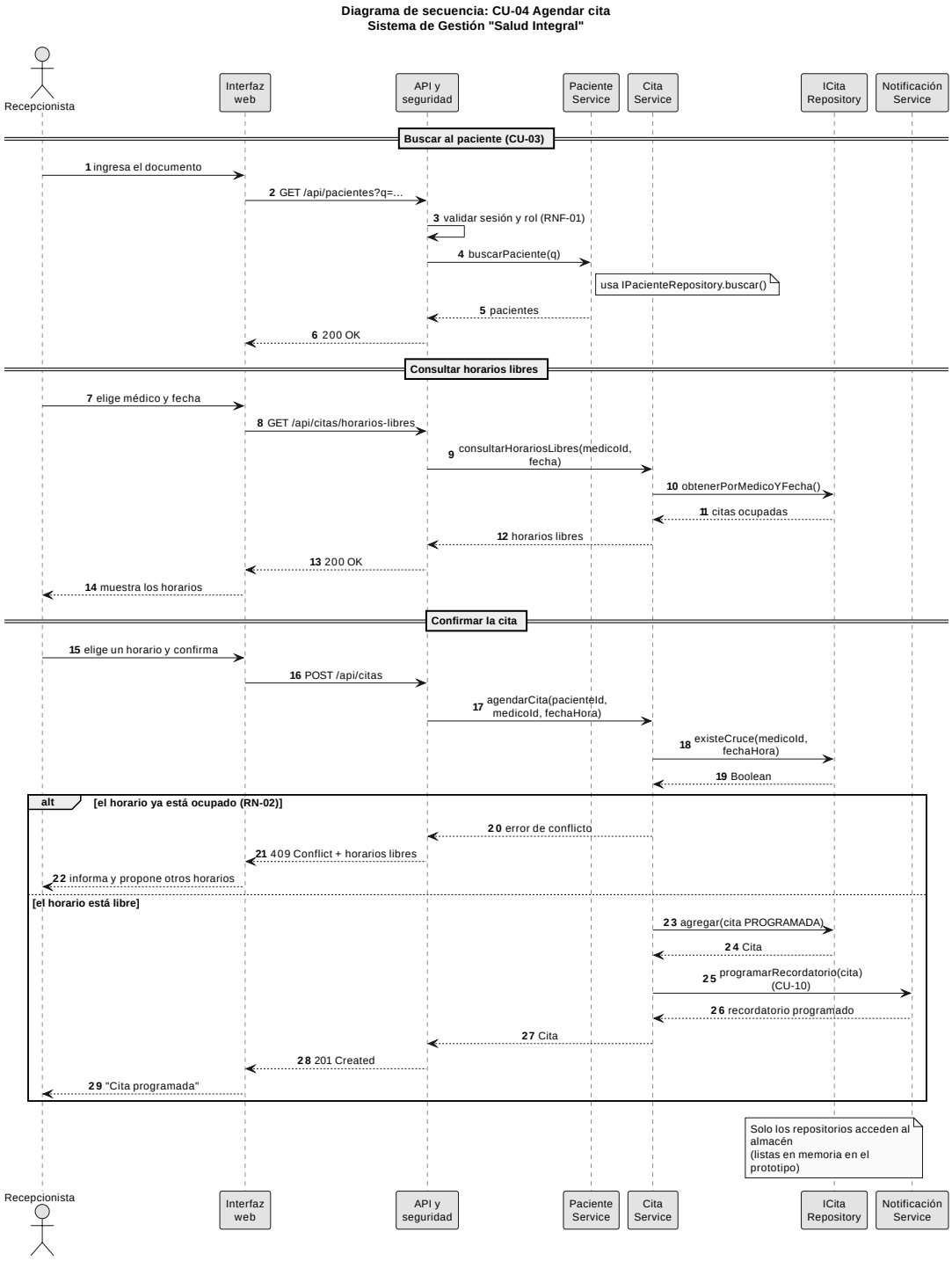
Nota. Elaboración propia con PlantUML. Fuente: `docs/uml/04_despliegue.puml`.

4.5 Secuencia: agendar una cita

Agendar una cita es el flujo que más dolores resuelve (D-02, D-03 y D-06), por eso se detalla en la Figura 5. La interfaz consulta primero al paciente y los horarios libres; al confirmar, *CitaService* pregunta al repositorio si ya existe un cruce (RN-02) y solo entonces guarda la cita y programa el recordatorio. Si el horario fue ocupado entretanto, la API responde 409 con horarios alternativos.

Figura 5

Diagrama de secuencia del caso de uso CU-04



Nota. Elaboración propia con PlantUML. Fuente: `docs/uml/03_secuencia_agendar_cita.puml`.

5. Patrones de diseño

5.1 Repository: el patrón principal

Repository actúa como una colección de objetos de dominio: ofrece operaciones como agregar, buscar y actualizar, y oculta cómo y dónde se guardan los datos (Evans, 2003; Fowler, 2002). Se eligió porque los dolores críticos de Salud Integral son dolores de datos (pérdida, duplicación, acceso indebido, falta de trazabilidad), y este patrón concentra su control en un único punto en lugar de repartirlo entre pantallas y servicios. Además respeta la regla de dependencia de la arquitectura limpia: el código de negocio apunta hacia abstracciones y no hacia detalles de almacenamiento (Martin, 2017).

En el sistema existe una interfaz genérica `IRepository[T]` con las operaciones comunes y una interfaz por entidad con las consultas propias del negocio. El fragmento siguiente es la interfaz real del prototipo:

```
class IRepository(ABC, Generic[T]):
    """Operaciones comunes. `desactivar` es un borrado lógico (RNF-11)."""
    @abstractmethod
    def agregar(self, entidad: T) -> T: ...
    @abstractmethod
    def obtener_por_id(self, id: int) -> Optional[T]: ...
    @abstractmethod
    def actualizar(self, entidad: T) -> T: ...
    @abstractmethod
    def desactivar(self, id: int) -> None: ...
    @abstractmethod
    def listar(self, incluir_inactivos: bool = False) -> list[T]: ...

class IPacienteRepository(IRepository[Paciente]):
    def obtener_por_documento(self, tipo: str, numero: str) -> Optional[Paciente]: ...
    def buscar(self, texto: str, limite: int = 50) -> list[Paciente]: ...
```

La implementación del prototipo, `MemoryRepository`, guarda *copias* de los objetos en una lista protegida por un candado: quien recibe un objeto no puede modificar el almacén sin llamar a `actualizar`, igual que ocurriría con una base de datos real. El repositorio de pacientes hereda de ella y añade la auditoría: cada lectura, creación o modificación queda anotada con el usuario de la petición (RF-13), sin que los servicios tengan que acordarse de hacerlo.

Tabla 4

Cómo el patrón Repository responde a los requisitos

Requisito o dolor	Cómo lo resuelve el patrón en el prototipo
RF-02 (D-02)	La verificación de documento único vive en el servicio de pacientes y se apoya en `obtener_por_documento`; ninguna pantalla puede saltársela
RNF-11 (D-01)	El borrado lógico se implementa una sola vez en `MemoryRepository.desactivar`; no existe operación de eliminación física
RF-13 (D-07)	La anotación en la bitácora se dispara desde los repositorios de pacientes y consultas, así que toda lectura o escritura clínica queda registrada
RNF-09	Los servicios dependen de interfaces; ninguna clase de negocio contiene acceso directo al almacén
RNF-10 / RNF-06	Cambiar de almacén (por ejemplo, a una base SQL) o agregar una caché a la búsqueda solo exige una nueva implementación
Pruebas	Los repositorios en memoria permiten probar las reglas de negocio sin conexión a la nube

5.2 Arquitectura por capas y composición de dependencias

El backend se organiza en capas: *API* (rutas, validación y permisos), *servicios* (reglas de negocio), *repositorios* (interfaces y almacenamiento) y *dominio* (entidades y errores de negocio). Un único archivo, `container.py`, es la raíz de composición: allí se decide qué implementación concreta de cada repositorio se usa y se inyecta en los servicios. Pasar a una base de datos SQL implicaría cambiar solo esas líneas.

5.3 Patrones de nube y seguridad aplicados

Health Endpoint Monitoring. La ruta `/health`, sin autenticación, comprueba que cada repositorio responde y devuelve 200 si todo está bien o 503 si algún componente falla, sin exponer datos de pacientes. Es lo que una plataforma en la nube o un balanceador consulta periódicamente para medir la disponibilidad (RNF-04) y reiniciar el servicio si deja de responder (Microsoft, s. f.-a).

Control de acceso basado en roles (RBAC). Cada ruta declara los roles permitidos mediante una dependencia reutilizable; recepción, médico y administrador ven solo lo que su cargo exige (Sandhu et al., 1996). Esto implementa RNF-01 y RN-03: la recepción no puede leer diagnósticos ni historias clínicas.

Notificador intercambiable. Los recordatorios usan una interfaz de notificación con una implementación simulada, de modo que conectar un proveedor real de correo o SMS no cambia la lógica de las citas, y el proceso que los envía corre en segundo plano cada minuto.

Tabla 5

Patrones de nube considerados para una versión de producción

Patrón	Problema que resolvería	Estado
Cache-Aside (Microsoft, s. f.-b)	Acelerar la búsqueda de pacientes cuando crezca el volumen (RNF-06)	No implementado
Queue-Based Load Leveling (Microsoft, s. f.-c)	Absorber picos de recordatorios enviándolos desde una cola en lugar de un ciclo	No implementado
Health Endpoint Monitoring (Microsoft, s. f.-a)	Monitorear la disponibilidad del servicio	Implementado (`/health`)

6. Prototipo

6.1 Tecnología y estructura

El backend usa Python y FastAPI (Ramírez, s. f.); el frontend es JavaScript con módulos ES, sin paso de compilación, y lo sirve la propia aplicación en la raíz. Como solicitó el equipo, la base de datos del prototipo es una **lista de objetos en memoria** accedida solo a través de los repositorios. El código se compone de unas 2 500 líneas de Python en la aplicación, 750 en las pruebas y 1 500 en el frontend, y se entrega con Dockerfile y configuración de despliegue para Render.

6.2 API REST

La aplicación expone 27 operaciones, documentadas automáticamente en `/docs`. La Tabla 6 las agrupa por módulo y rol.

Tabla 6

Operaciones de la API por módulo

Módulo	Operaciones principales	Roles
Sesión	`POST /api/auth/login`, `GET /api/auth/me`, `POST /api/auth/logout`	Todos
Pacientes	`POST/GET /api/pacientes`, `GET/PUT /api/pacientes/{id}`	Recepción; búsqueda también médico
Historia clínica	`GET /api/pacientes/{id}/historial`	Médico
Citas	`GET /api/citas/horarios-libres`, `GET /api/citas/agenda`, `POST /api/citas`, `PUT …/reprogramar`, `POST …/cancelar`	Recepción; agenda también médico (la suya)
Consultas	`POST /api/consultas`, `GET/PUT /api/consultas/{id}`, `POST …/cerrar`, `POST …/notas`, `GET …/por-cita/{id}`	Médico
Usuarios	`GET/POST /api/usuarios`, `POST …/desactivar`	Administrador
Auditoría y recordatorios	`GET /api/auditoria`, `GET /api/recordatorios`, `POST …/procesar`	Administrador (recordatorios también recepción)
Monitoreo	`GET /health`	Público

6.3 Seguridad y reglas de negocio

Las contraseñas se guardan con PBKDF2-SHA256 y 200 000 iteraciones (Moriarty et al., 2017). La sesión se representa con un token JWT firmado (Jones et al., 2015) que vence a los 30 minutos y puede revocarse al cerrar sesión; las respuestas de la API llevan `Cache-Control: no-store`. Los mensajes de error de inicio de sesión no revelan si falló el usuario o la contraseña, y los intentos fallidos quedan en la bitácora. En el frontend todo texto proveniente del servidor se inserta como texto, nunca como HTML, para evitar inyección de código.

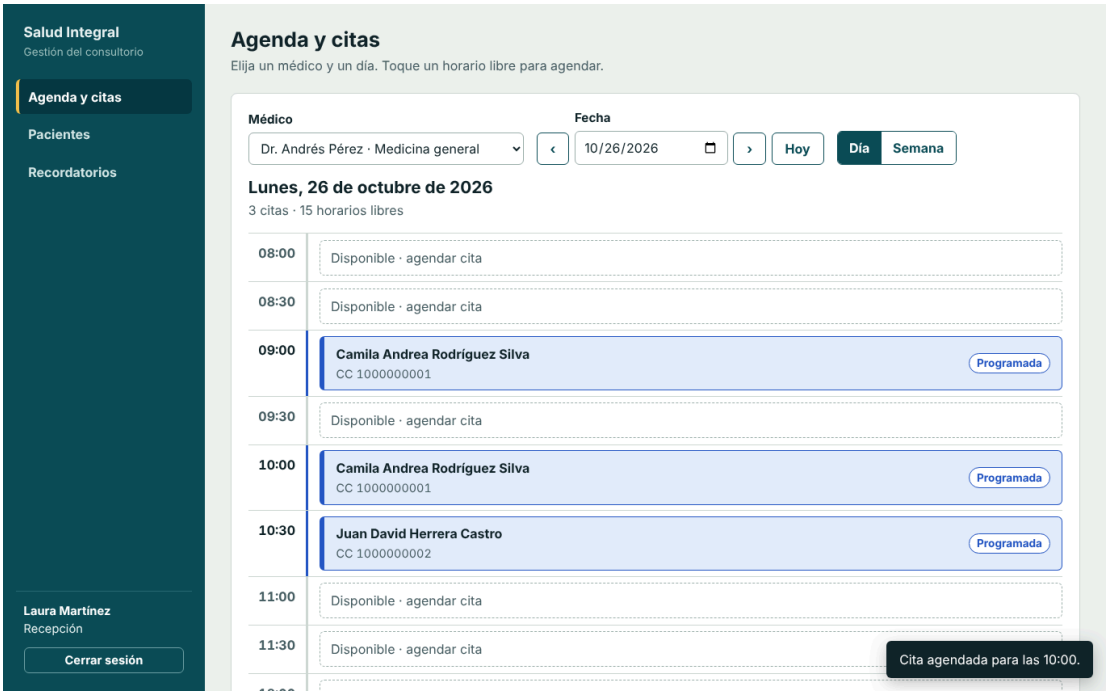
Las reglas críticas se aplican en el servidor, no en la pantalla. El cruce de horarios (RN-02) se evalúa y se guarda dentro de una misma sección protegida por candado, de modo que dos recepcionistas que intenten el mismo horario a la vez no pueden duplicarlo; no se aceptan citas en el pasado ni fuera de la jornada (08:00 a 17:00 en franjas de 30 minutos, configurable); una consulta cerrada rechaza ediciones y solo admite notas aclaratorias (RN-04); y cancelar libera el horario de inmediato (RN-06).

6.4 Interfaz de usuario

La interfaz es una aplicación de una sola página con un menú distinto por rol. La pieza central es la agenda del día como línea de tiempo (Figura 6): cada franja de 30 minutos aparece libre (se agenda con un clic) u ocupada (se abre para reprogramar o cancelar), lo que evita los cruces antes de que ocurran. El médico ve su agenda, atiende la cita en un formulario con la alerta de alergias a la vista y el historial al lado, y al cerrar la consulta esta queda protegida (Figura 7). El administrador dispone de la gestión de usuarios y de la bitácora de auditoría (Figura 8).

Figura 6

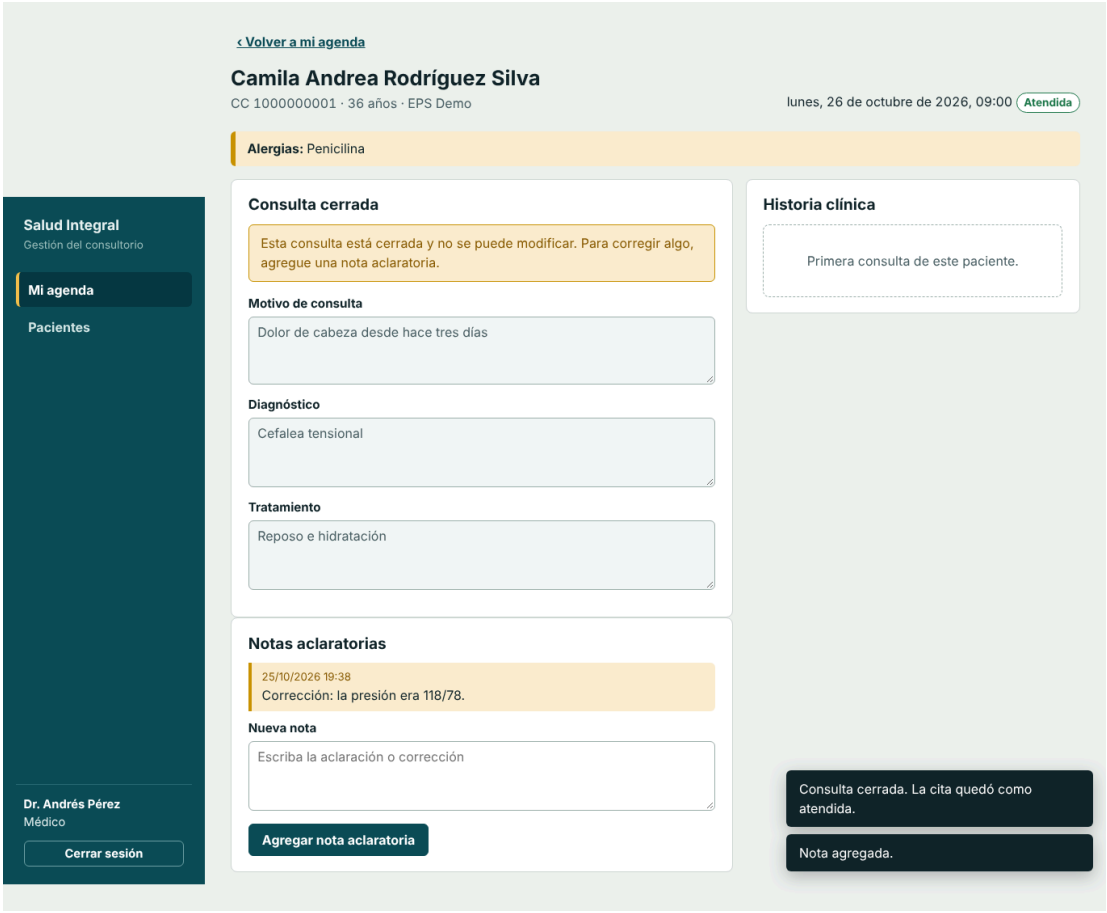
Agenda del día de un médico vista por recepción



Nota. Captura del prototipo. Los pacientes son ficticios.

Figura 7

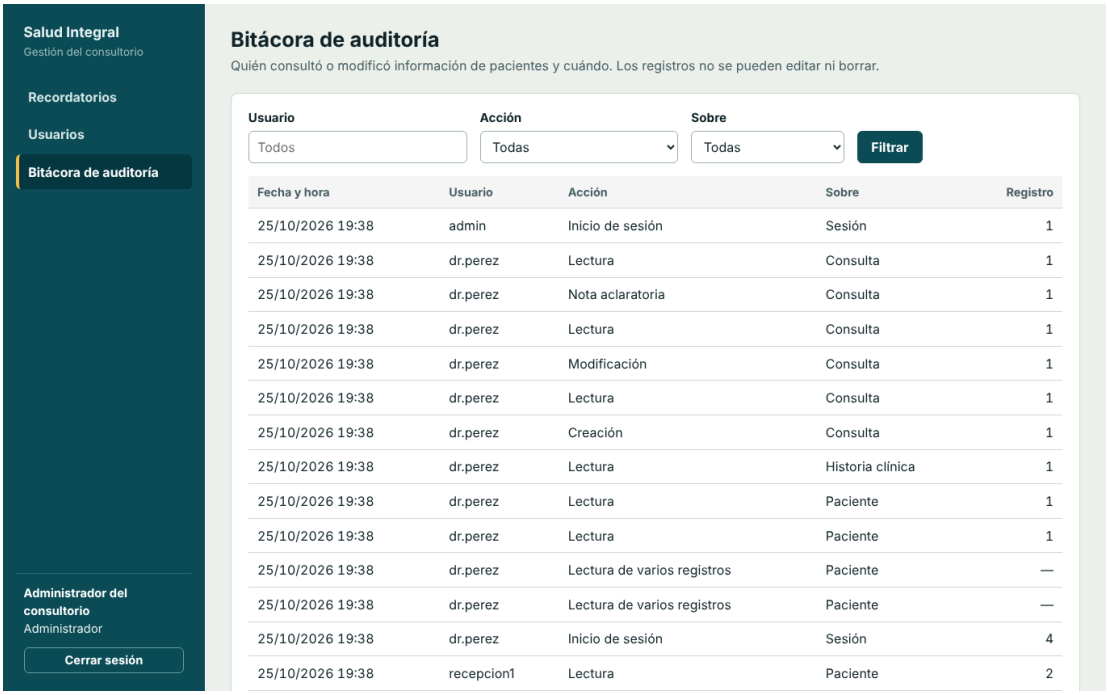
Consulta cerrada con nota aclaratoria e historial del paciente



Nota. Captura del prototipo en la pantalla del médico.

Figura 8

Bitácora de auditoría del administrador



Nota. Captura del prototipo. Cada fila indica quién hizo qué, sobre qué registro y cuándo.

6.5 Pruebas y resultados

El prototipo se verificó con 57 pruebas automáticas (pytest) que ejercitan la API completa: contrato de los repositorios (copias, borrado lógico, búsqueda sin tildes, auditoría), unicidad de documento, cruce y reprogramación de citas, recordatorios, flujo de consulta con cierre y notas, permisos por rol, expiración y revocación de sesión, gestión de usuarios y contenido de la bitácora. Todas pasan. Además, los tres flujos de usuario (recepción, médico y administrador) se recorrieron de extremo a extremo en un navegador Chromium. El rendimiento de la búsqueda se midió con 10 000 pacientes cargados en el repositorio en memoria: una búsqueda por documento o por nombre tardó entre 32 y 42 ms (mediana de cinco ejecuciones), muy por debajo de los 2 s exigidos.

Tabla 7

Criterios de éxito: situación inicial y resultado en el prototipo

Indicador	Situación actual	Meta	Resultado
Búsqueda de un paciente	Varios minutos	Menos de 2 s	32–42 ms con 10 000 pacientes
Pacientes duplicados	Existen	Ninguno	Rechazados por tipo y número de documento
Cruces de horario	Frecuentes	Ninguno	Imposibles (409 con horarios alternativos)
Citas con recordatorio	Ninguna	Todas	Todas se programan 24 h antes (simulado)
Accesos con rol y registro	Ninguno	Todos	Todas las rutas con rol; lecturas y cambios auditados
Copias de seguridad	Inexistentes	Diarias automáticas	No cubierto: los datos viven en memoria

6.6 Cobertura de requisitos y límites

Se cumplen 12 de los 14 requisitos funcionales. **No se implementaron** RF-09 (adjuntar documentos) y RF-14 (reportes), ambos de prioridad no crítica. RF-10 se cumple de forma simulada: el recordatorio se programa y se "envía" en el sistema, pero no sale ningún correo real. RF-12 cierra la sesión a los 30 minutos de iniciada, no por inactividad. Entre los requisitos no funcionales, el cifrado en tránsito (RNF-02) depende de que la plataforma de despliegue sirva la aplicación por HTTPS; la disponibilidad (RNF-04) cuenta con el endpoint de salud pero no se midió en el tiempo; y los respaldos (RNF-05) no se cubren porque los datos están en memoria y se restablecen con los datos de demostración al reiniciar. La escalabilidad (RNF-07) está limitada a un solo proceso, ya que varios procesos tendrían cada uno sus propios datos. La usabilidad (RNF-08) no se evaluó con usuarios reales.

6.7 Despliegue

El repositorio incluye un `Dockerfile` y un `render.yaml` para publicar el prototipo como servicio web en Render: la plataforma construye la imagen, genera la clave secreta y usa `/health` para verificar el servicio. El prototipo se publicó en esa plataforma y está disponible en **https://consultorio-medico-pty2.onrender.com**. En el plan gratuito el servicio se suspende tras un periodo sin uso, por lo que la primera visita puede tardar unos segundos, y los datos se restablecen cada vez que se reinicia. La ejecución local requiere solo instalar las dependencias y arrancar `uvicorn`; las instrucciones y los usuarios de demostración están en el archivo README del repositorio.

7. Conclusiones

El problema se resolvió en lo esencial. El prototipo cubre los flujos que sostienen el trabajo diario del consultorio (registrar pacientes sin duplicados, agendar sin cruces, atender y cerrar consultas, controlar el acceso y auditarlo) y todos sus requisitos de prioridad Must están implementados y probados, con la salvedad sobre la sesión por inactividad que se explica en la sección 6.6.

Repository concentró el control de los datos. Al ubicar la unicidad, el borrado lógico y la auditoría en la capa de repositorios, las reglas se aplican igual desde cualquier pantalla o ruta, y los servicios quedaron probables sin base de datos. La decisión más rentable fue definir las interfaces antes que las implementaciones: permitió empezar con listas en memoria, como pidió el equipo, sin hipotecar el paso a una base de datos real.

Lo que el prototipo no demuestra. No hay persistencia ni respaldos, que eran el dolor D-04; tampoco adjuntos ni reportes, y el envío de recordatorios es simulado. Por eso el sistema es una demostración de arquitectura y no un producto listo para guardar historias clínicas reales.

Trabajo futuro. En orden de prioridad: (1) una implementación de los repositorios sobre una base de datos administrada con respaldos automáticos; (2) adjuntar documentos y reportes (RF-09 y RF-14); (3) un proveedor real de correo o SMS, con una cola para absorber picos; (4) una caché para la búsqueda de pacientes; y (5) una evaluación de usabilidad con el personal del consultorio.

Referencias

Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

ISO/IEC/IEEE. (2018). *Systems and software engineering — Life cycle processes — Requirements engineering* (ISO/IEC/IEEE 29148:2018).

Jones, M., Bradley, J., y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Microsoft. (s. f.-a). *Health endpoint monitoring pattern*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/patterns/health-endpoint-monitoring

Microsoft. (s. f.-b). *Cache-aside pattern*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/patterns/cache-aside

Microsoft. (s. f.-c). *Queue-based load leveling pattern*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/patterns/queue-based-load-leveling

Ministerio de Salud de Colombia. (1999). *Resolución 1995 de 1999: Por la cual se establecen normas para el manejo de la historia clínica*.

Moriarty, K., Kaliski, B., y Rusch, A. (2017). *PKCS #5: Password-based cryptography specification version 2.1* (RFC 8018). Internet Engineering Task Force. https://doi.org/10.17487/RFC8018

Ramírez, S. (s. f.). *FastAPI* [Marco de desarrollo]. https://fastapi.tiangolo.com

Sandhu, R. S., Coyne, E. J., Feinstein, H. L., y Youman, C. E. (1996). Role-based access control models. *Computer, 29*(2), 38–47. https://doi.org/10.1109/2.485845

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.

Wiegers, K., y Beatty, J. (2013). *Software requirements* (3.ª ed.). Microsoft Press.

[Agregar aquí la bibliografía del curso utilizada para la selección de patrones.]